CS151 Intro to Data Structures

Heaps

Announcements

- HW5 and Lab7 released
  - due monday March 29th
  - Only one submission link on gradescope

- CS Major info session Wednesday 4-5pm
  - Park 231

Outline

1. Tree review
2. Heaps
   a. Definition
   b. Operations
   c. Implementing a heap

Warmup

```
    15
   / \
  6   23
 / \  /
3  10 18
  /
 8
```

1. Is this a valid BST?
2. Trace the search for 8. List each node visited in order
3. Trace the search for 11. List each node visited in order
4. Draw the tree after inserting 12. Briefly justify where it lands.
5. Starting from the original tree, remove 3. Draw the result
6. Starting from the original tree, remove 6. Draw the result and identify the removal case.
7. Suppose values were inserted in the order 3, 6, 8, 10, 15, 18, 23. Draw the resulting BST. What is its height? What does this tell you about BST runtime guarantees?

Warmup 2

```
    15
   /  \
  6    23
 / \   /
3  10 18
  /
 8
```

List the in-order, pre-order, and post-order traversals.

Heaps

Heaps

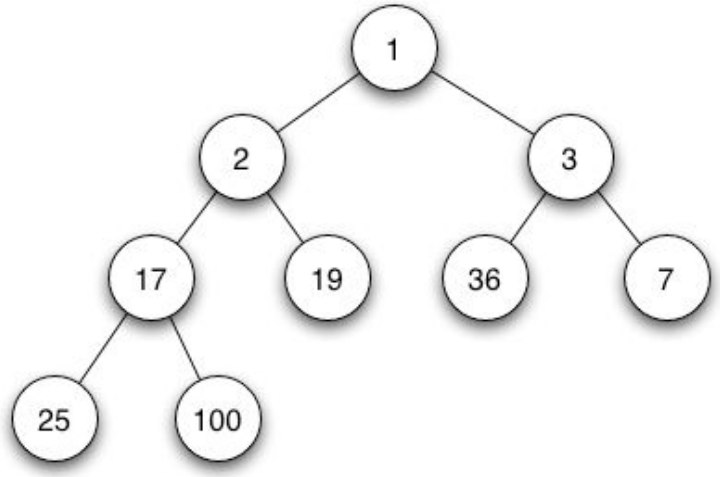
Allows you to efficiently access and modify the element with the **lowest value**

Binary Heap Properties

1. Each node has **at most 2 children**

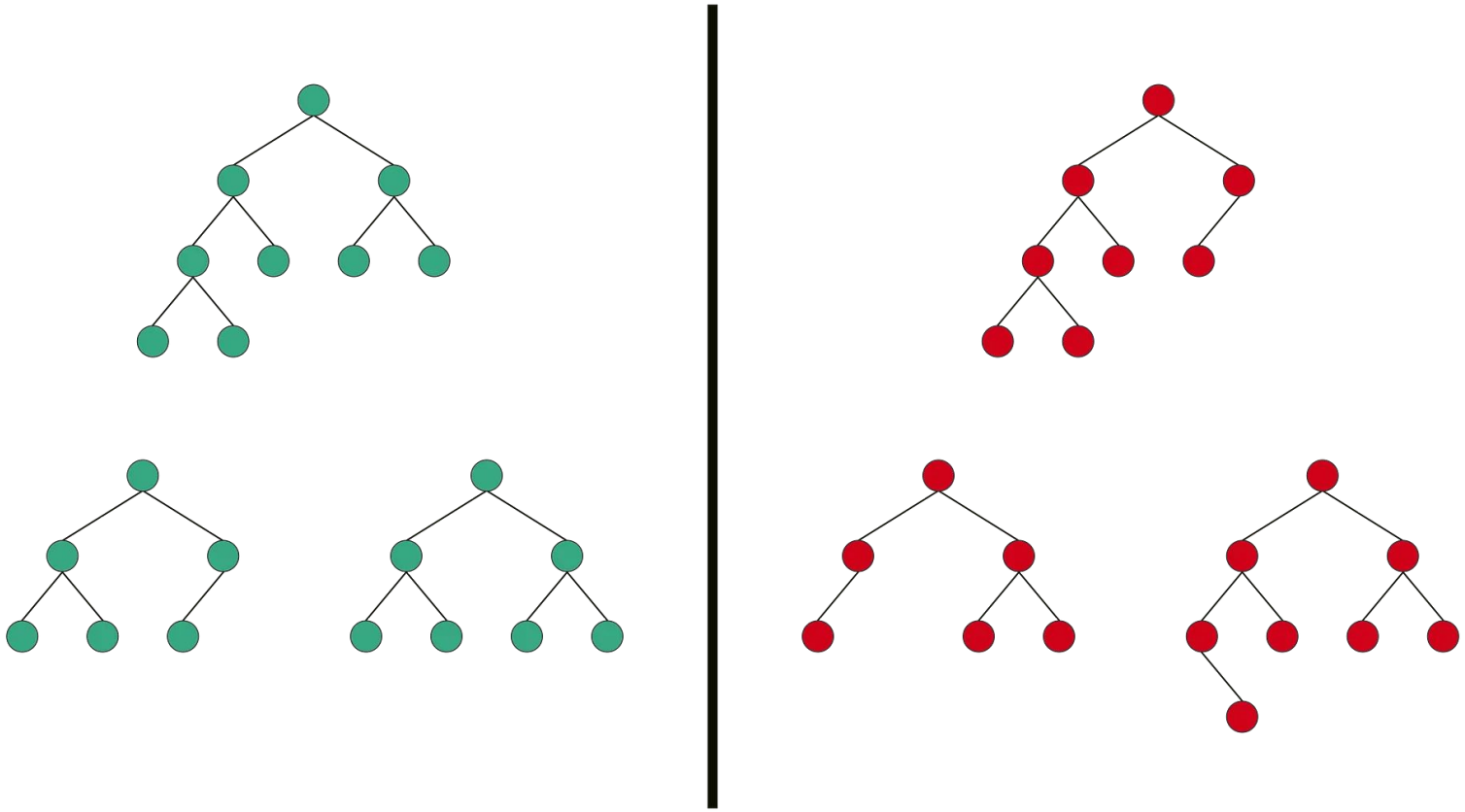
1. For every node n (except for the root): **n.key >= parent(n).key**

1. **Complete:** all levels of the tree, except possibly the last one are fully filled, and, if the last level of the tree is not fully filled, the nodes of that level are filled from left to right

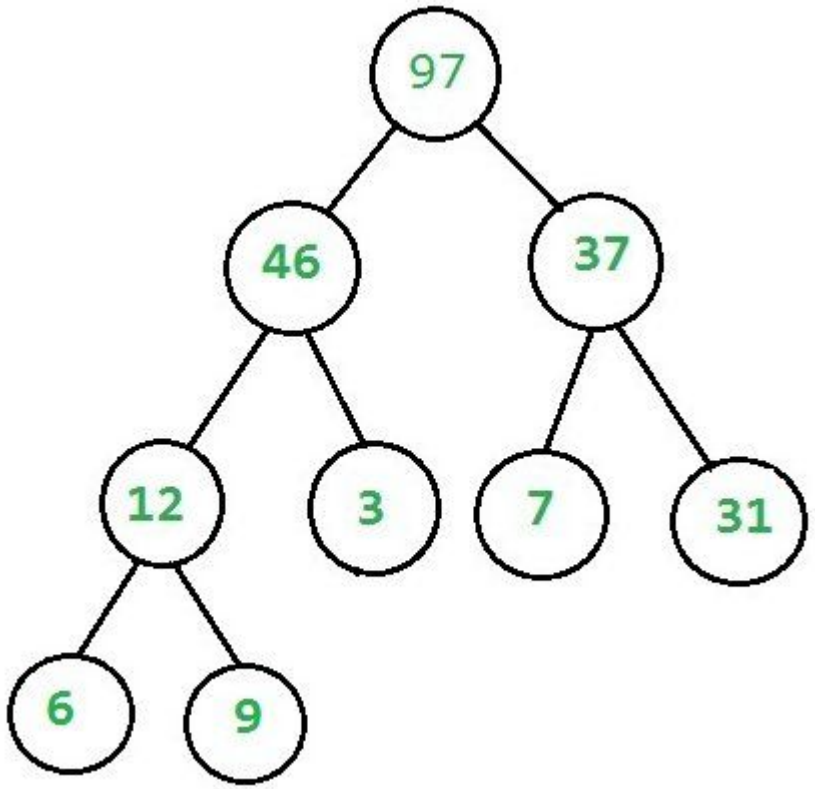


Complete Binary Trees

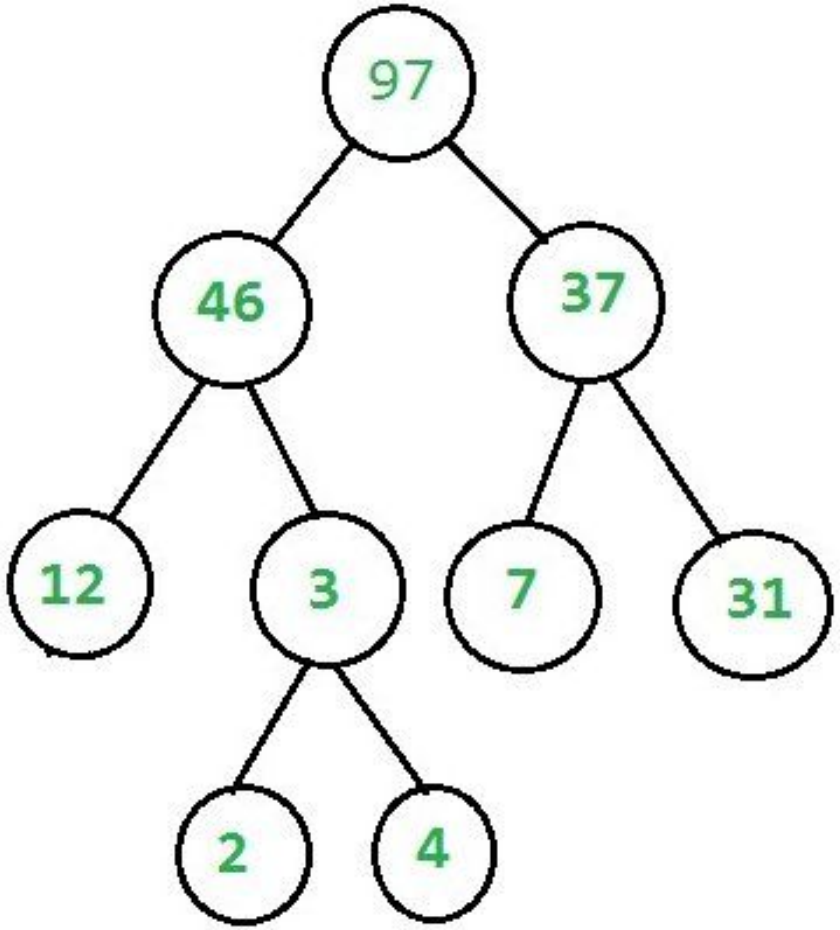
All levels of the tree, except possibly the last one are fully filled, and, if the last level of the tree is not complete, the nodes of that level are filled from left to right



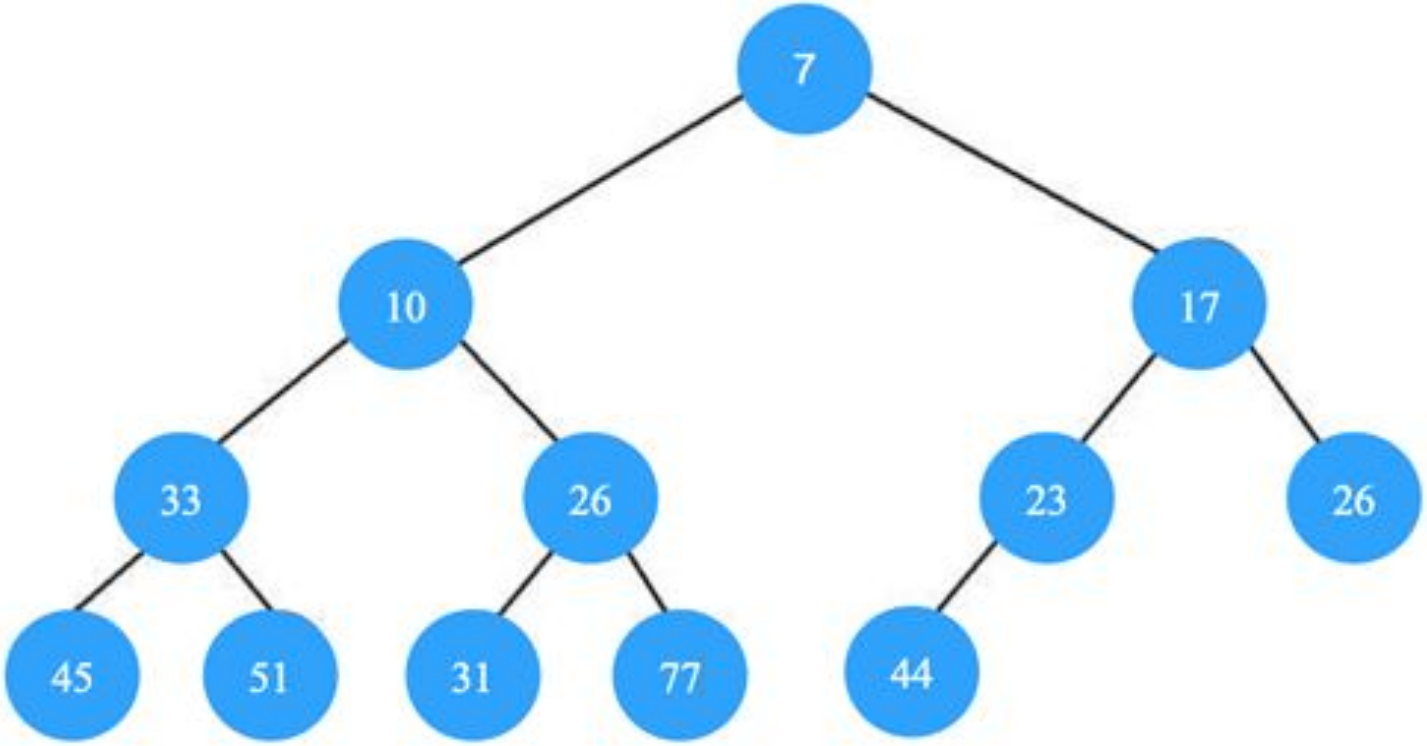
Is this a binary min heap?



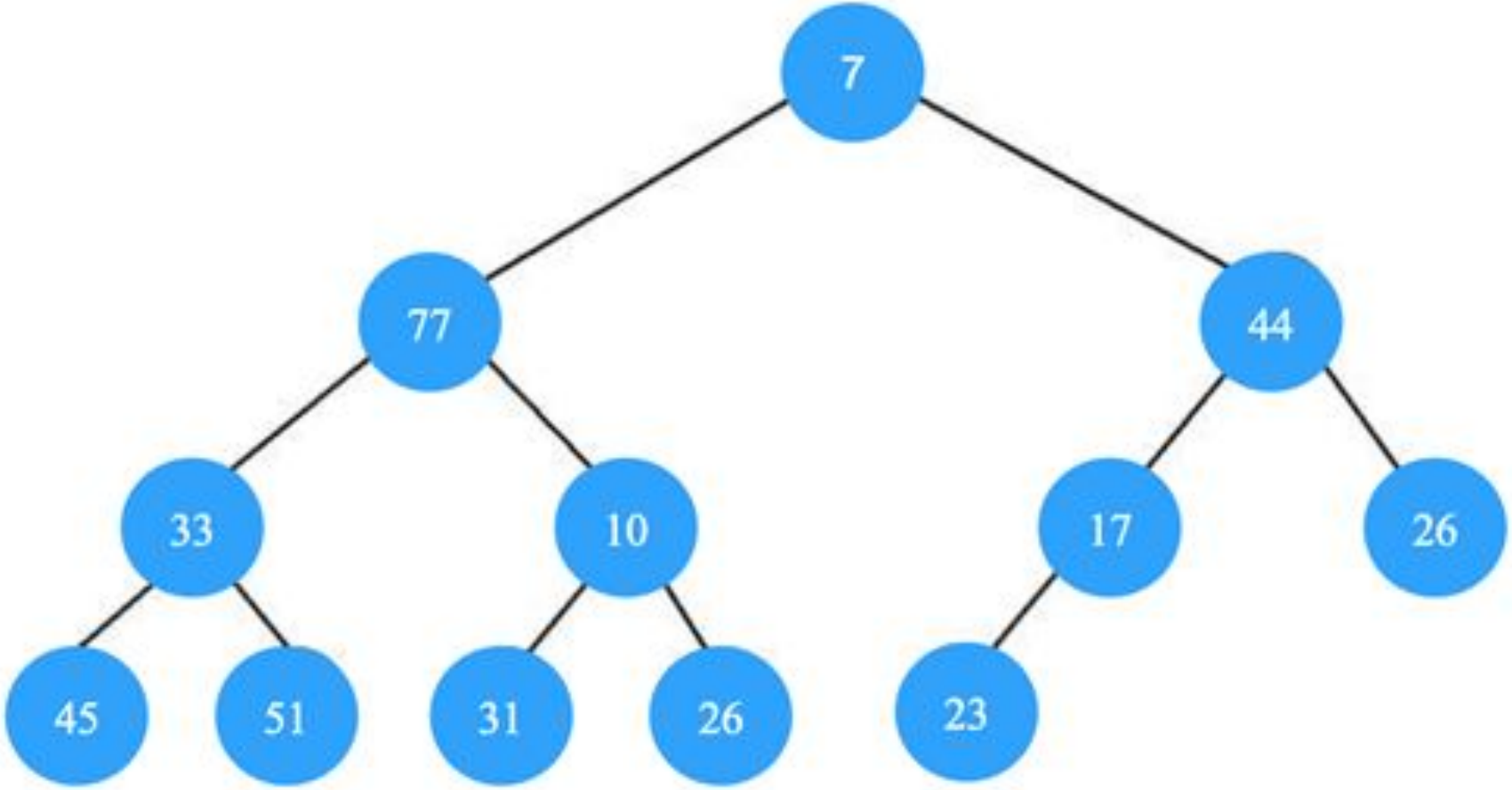
Is this a binary min heap?



Is this a binary min heap?

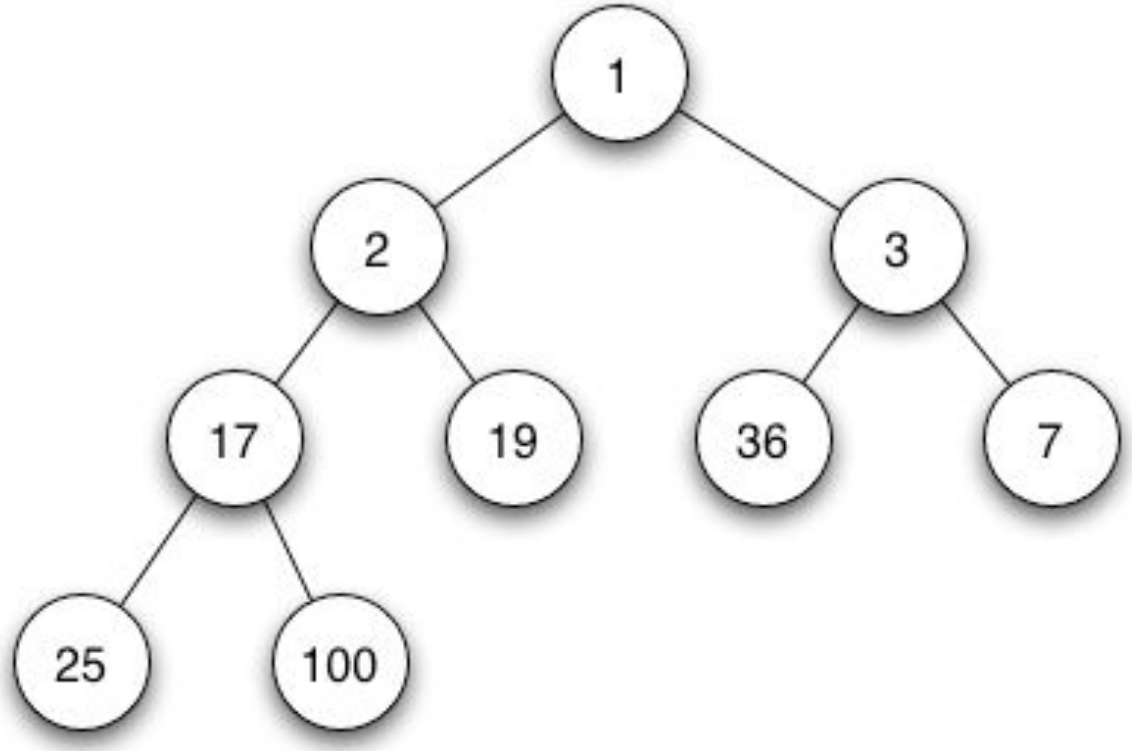


Is this a binary min heap?



Heap Properties

- Height?
  - log(n)



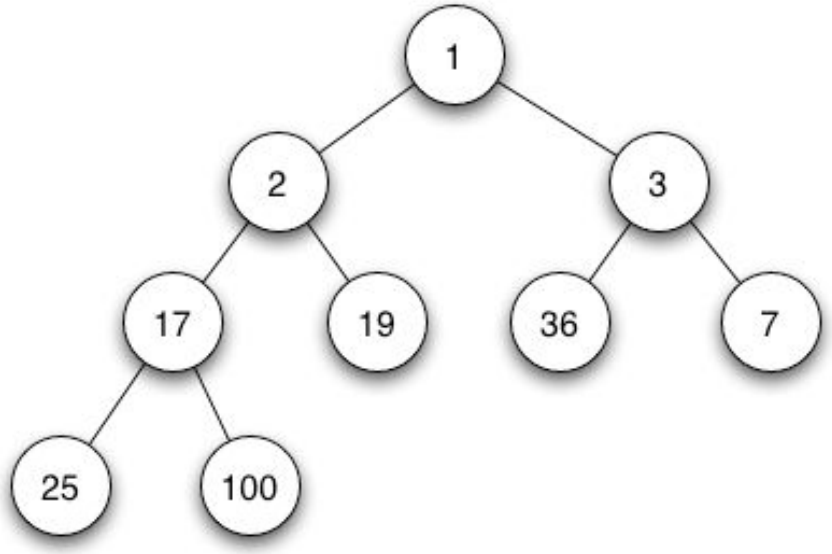
Heap Operations:

1. Insert
   a. Upheap
2. Poll
   a. Downheap
3. Remove
4. Search

Heap Insertion

Need to maintain:

1. completeness
2. key order property: **n.key >= parent(n).key**



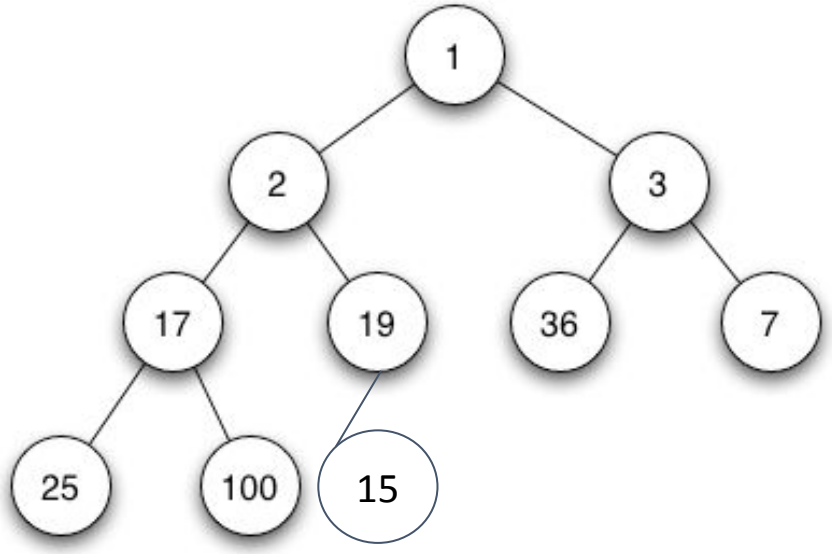

```
insert(15)
```

Heap Insertion

Need to maintain:

UPHEAP!

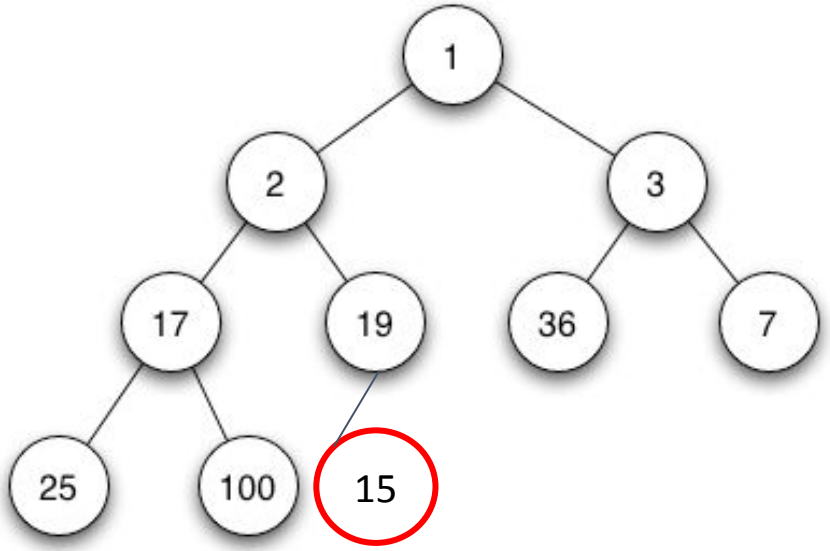
1. completeness
2. key order property: **n.key >= parent(n).key**

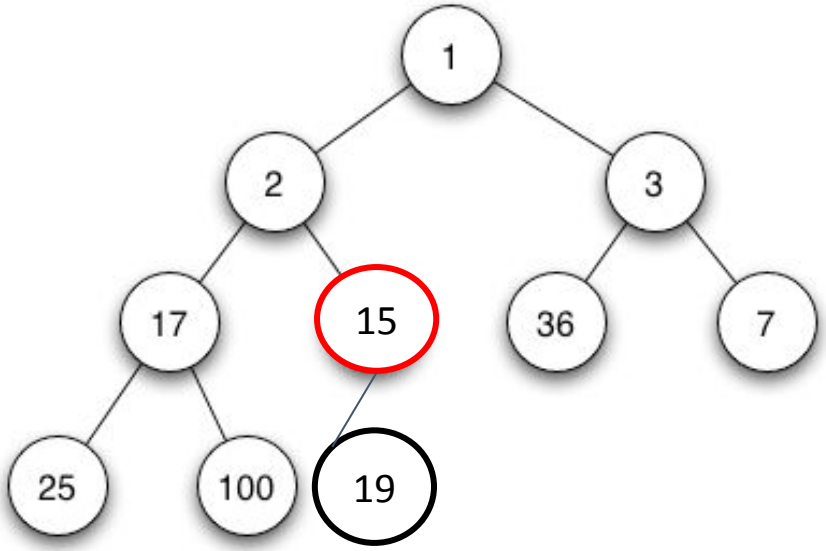



```
insert(15)
```

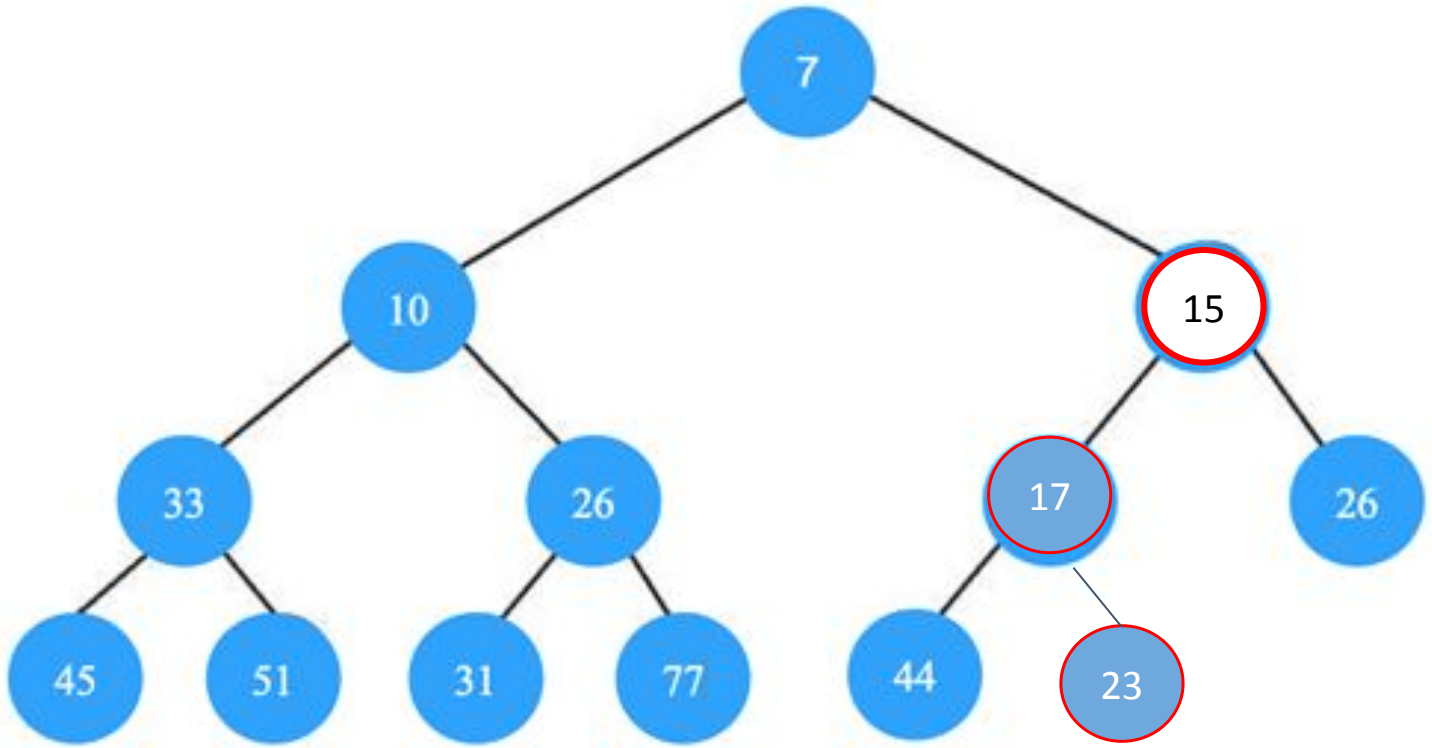
Heap Insertion - Upheap

1. Compare the added element with its parent; if they are in the correct order, stop.
2. If not, swap the element with its parent and return to the previous step.





Heap Insertion




```
insert(15)
```

Heap insertion

1. Add the element to the bottom level of the heap at the leftmost open space.
2. Compare the added element with its parent; if they are in the correct order, stop.
3. If not, swap the element with its parent and return to the previous step.

Runtime complexity?

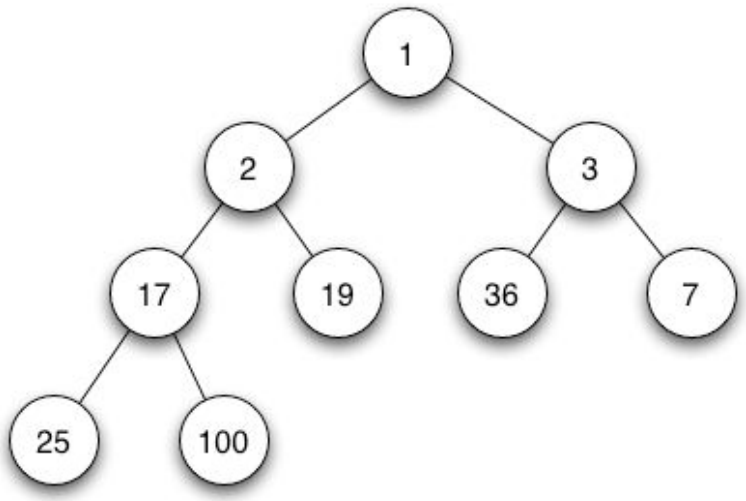
O(logn)

What about for an array implementation?

O(logn) assuming expansion is not necessary

Heap **Poll**

- Removing the root
- Also called extract



What properties do we need to maintain?

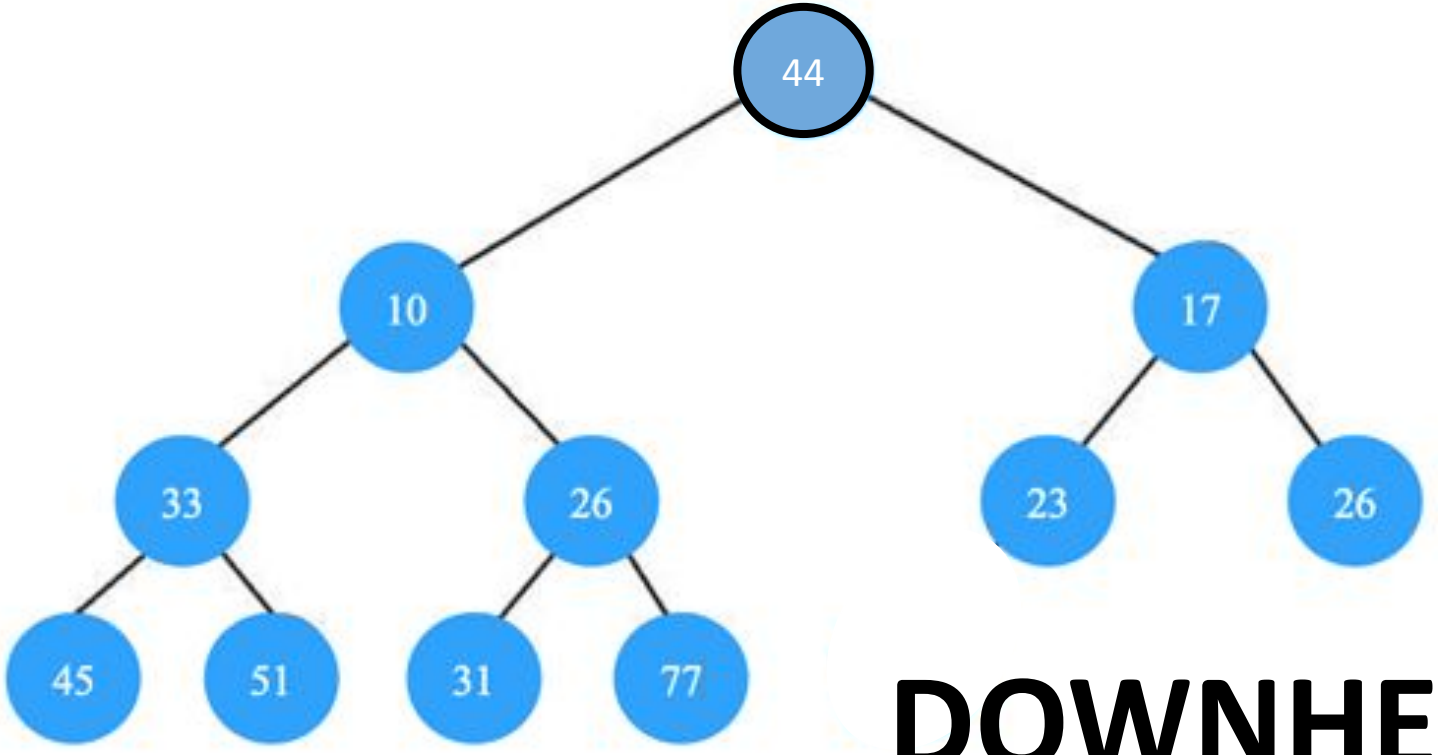
1. completeness
2. key order property: **n.key >= parent(n).key**

Heap Poll

1. Replace the root of the heap with the last element on the last level.
2. Compare the new root with its children; if they are in the correct order, stop.
3. If not, swap the element with one of its smallest children and return to the previous step

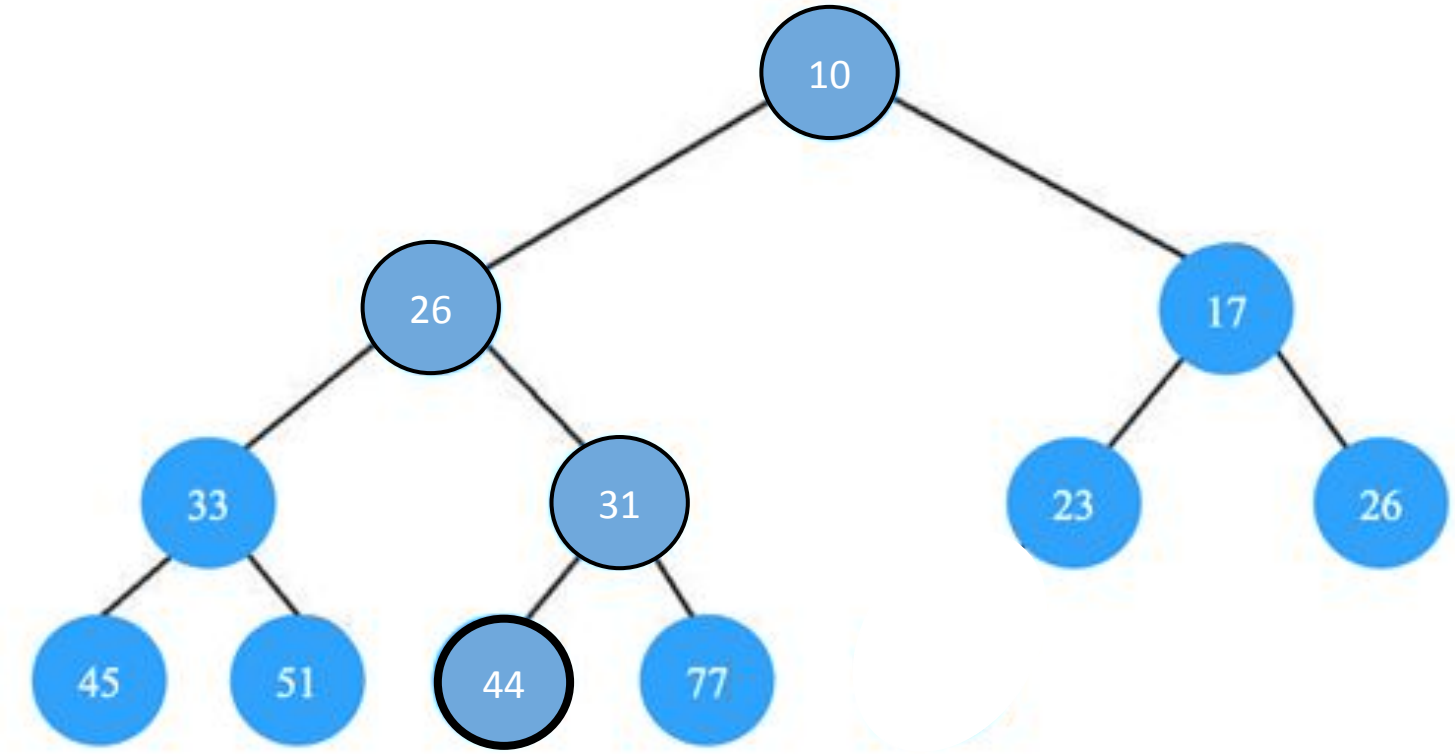
Heap Poll Example

1. Replace the root of the heap with the last element on the last level.



DOWNHEAP!

1. Compare the new root with its children; if they are in the correct order, stop.
2. If not, swap the element with one of its smallest children and return to the previous step



Heap Poll

1. Replace the root of the heap with the last element on the last level.
2. Compare the new root with its children; if they are in the correct order, stop.
3. If not, swap the element with one of its smallest children and return to the previous step

Runtime?

O(logn)

Array Implementation?

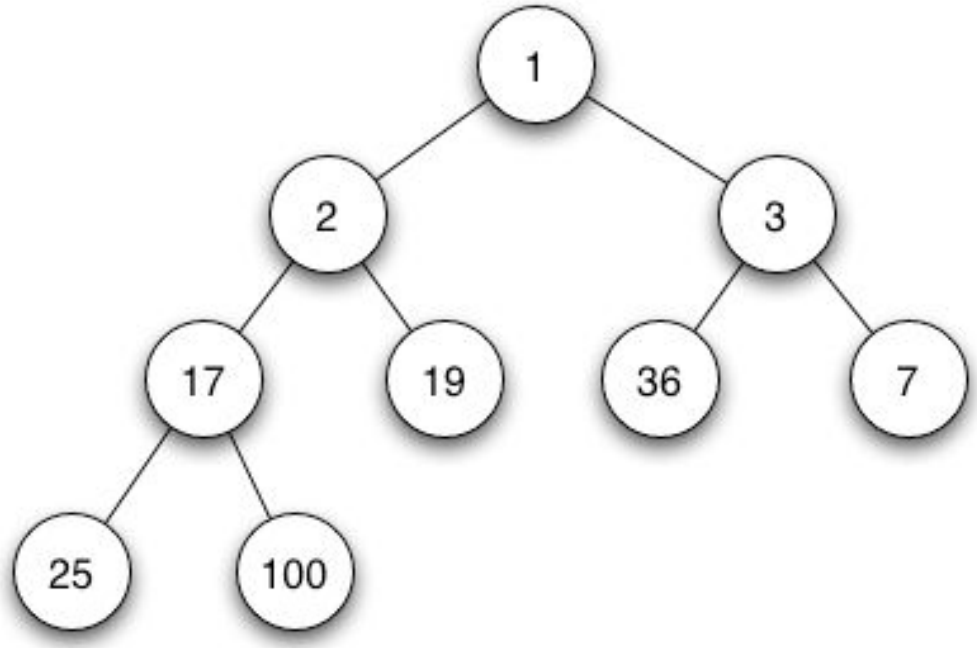
O(logn) - no expansions necessary for poll

Heap Search

- Is the element 36 in the heap?

- not as efficient as searching a BST
  - Weaker ordering property!
  - Best we can do is check every node
  - Shortcuts?

- Runtime complexity?
  - O(n)

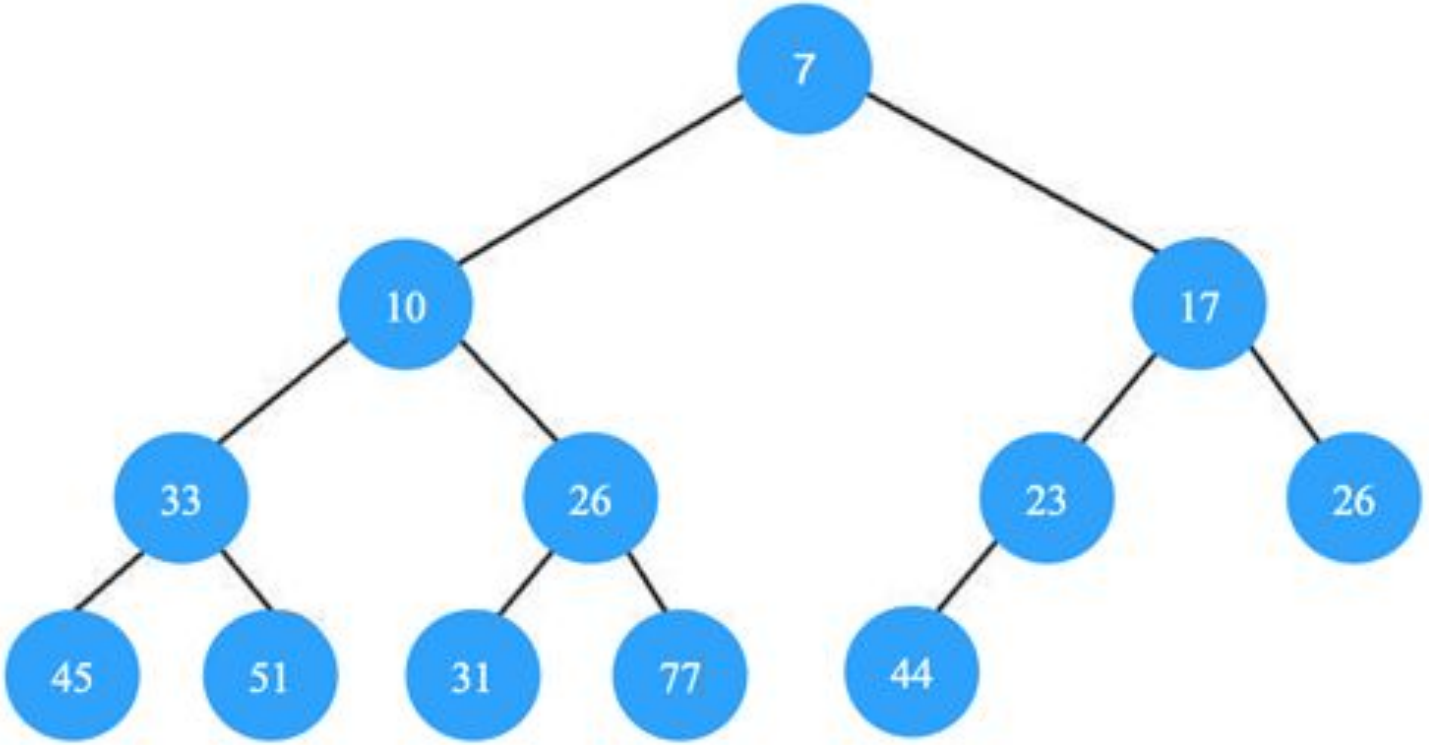


Heap Search

- Is the element 11 in the heap?

- Shortcuts?
  - Can stop once we hit

- Runtime complexity?
  - O(n)



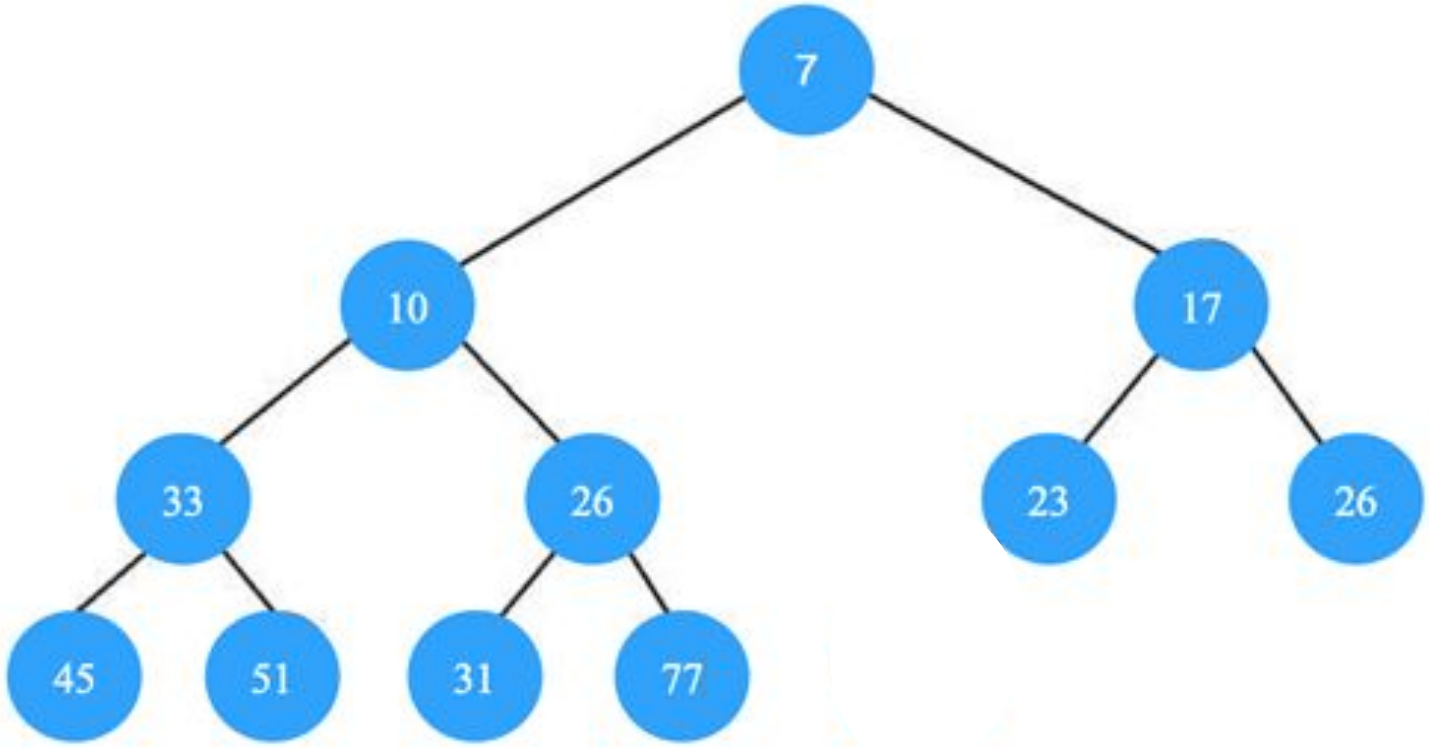
Heap **Remove**

More general case of poll

1. Find the index i of the element we want to delete
2. Swap this element with the last element (rightmost leaf)
3. Downheap to restore the heap property

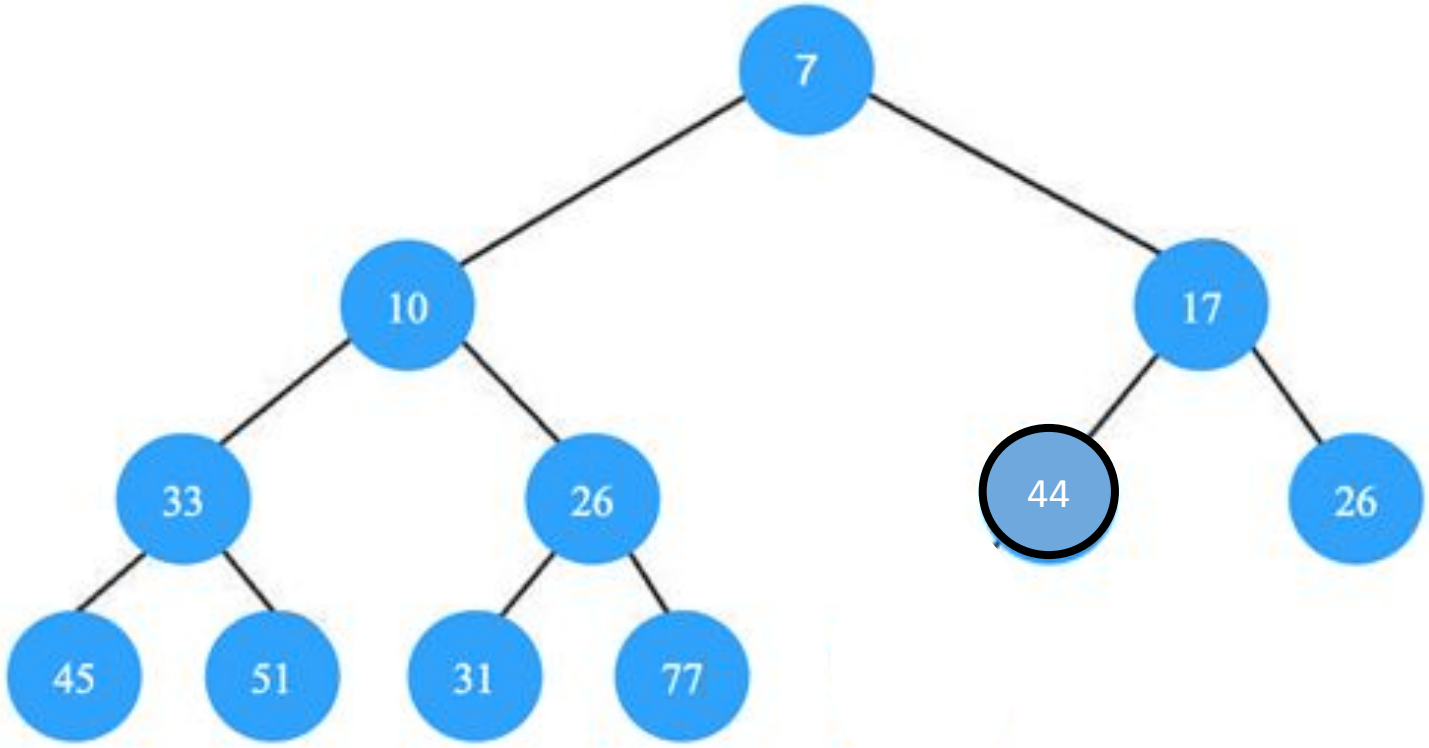
Heap Remove: Example 1

`remove(44)`



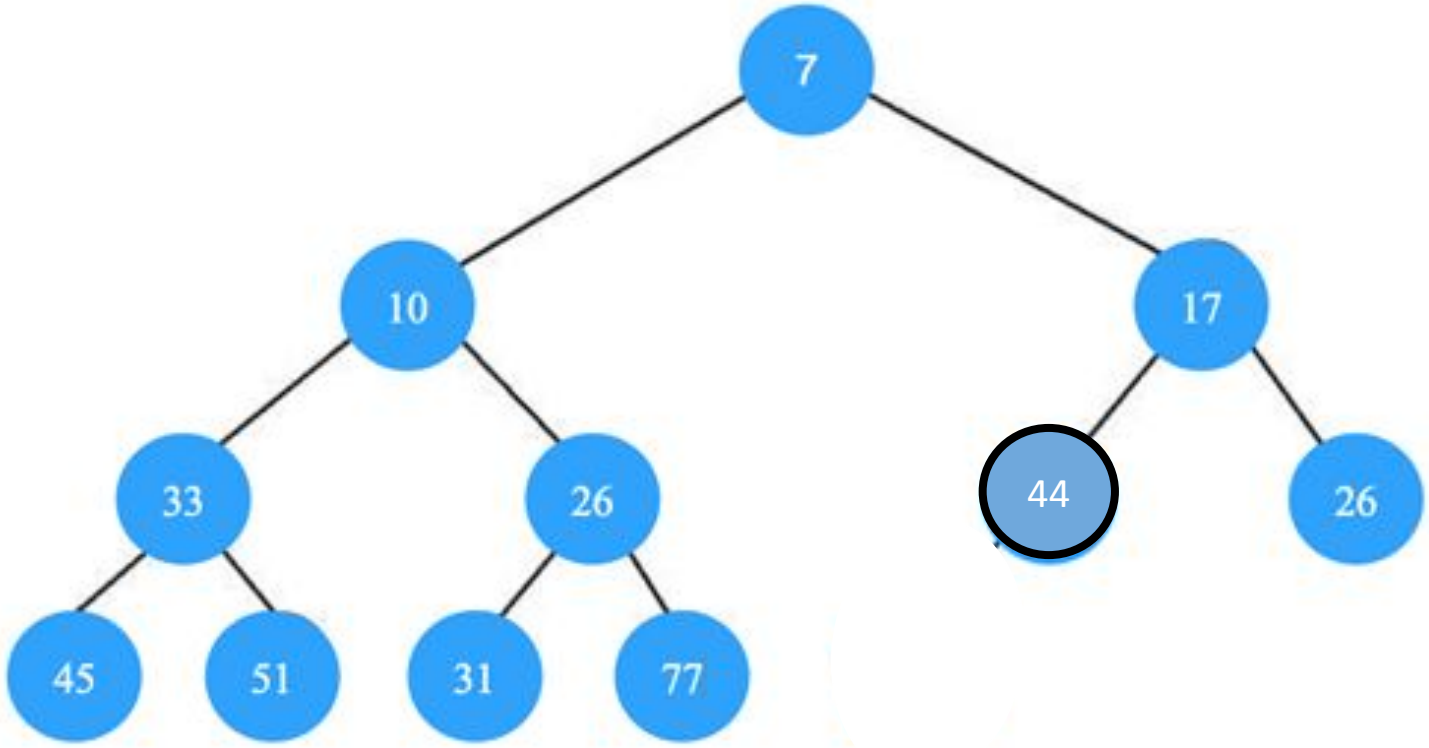
Heap Remove: Example 2

`remove(23)`



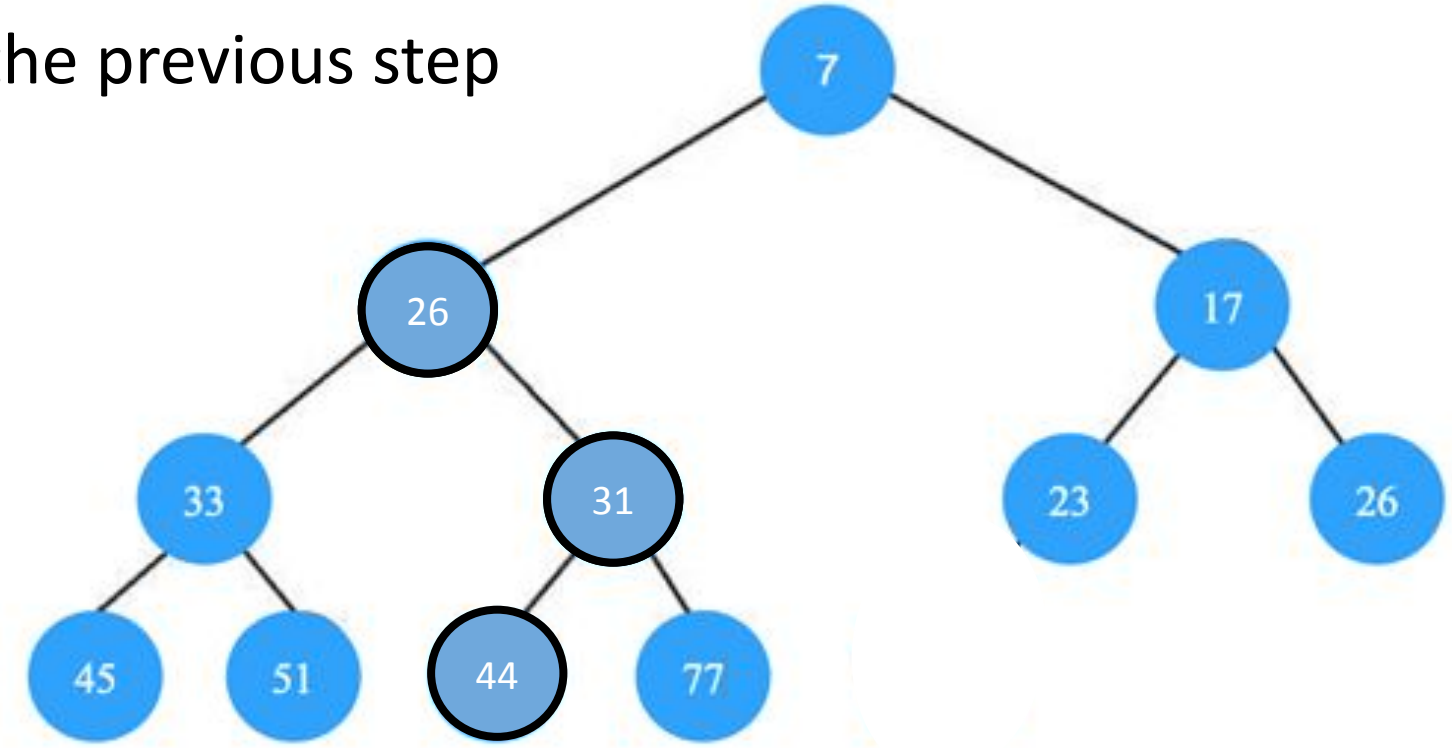
Heap Remove: Example 2

`remove(23)`



Downheap:

1. Compare the new root with its children; if they are in the correct order, stop.
2. If not, swap the element with one of its smallest children and return to the previous step



`remove(10)`

Heap **Remove**

More general case of poll

1. Find the index i of the element we want to delete
2. Swap this element with the last element
3. Downheap to restore the heap property

Runtime complexity?

O(n + logn)

Summary

1. Heaps
   a. DS for efficient removal of the min element
   b. Can be implemented as an array or node based
   c. logarithmic insertion, poll
   d. linear search
   e. log-linear removal